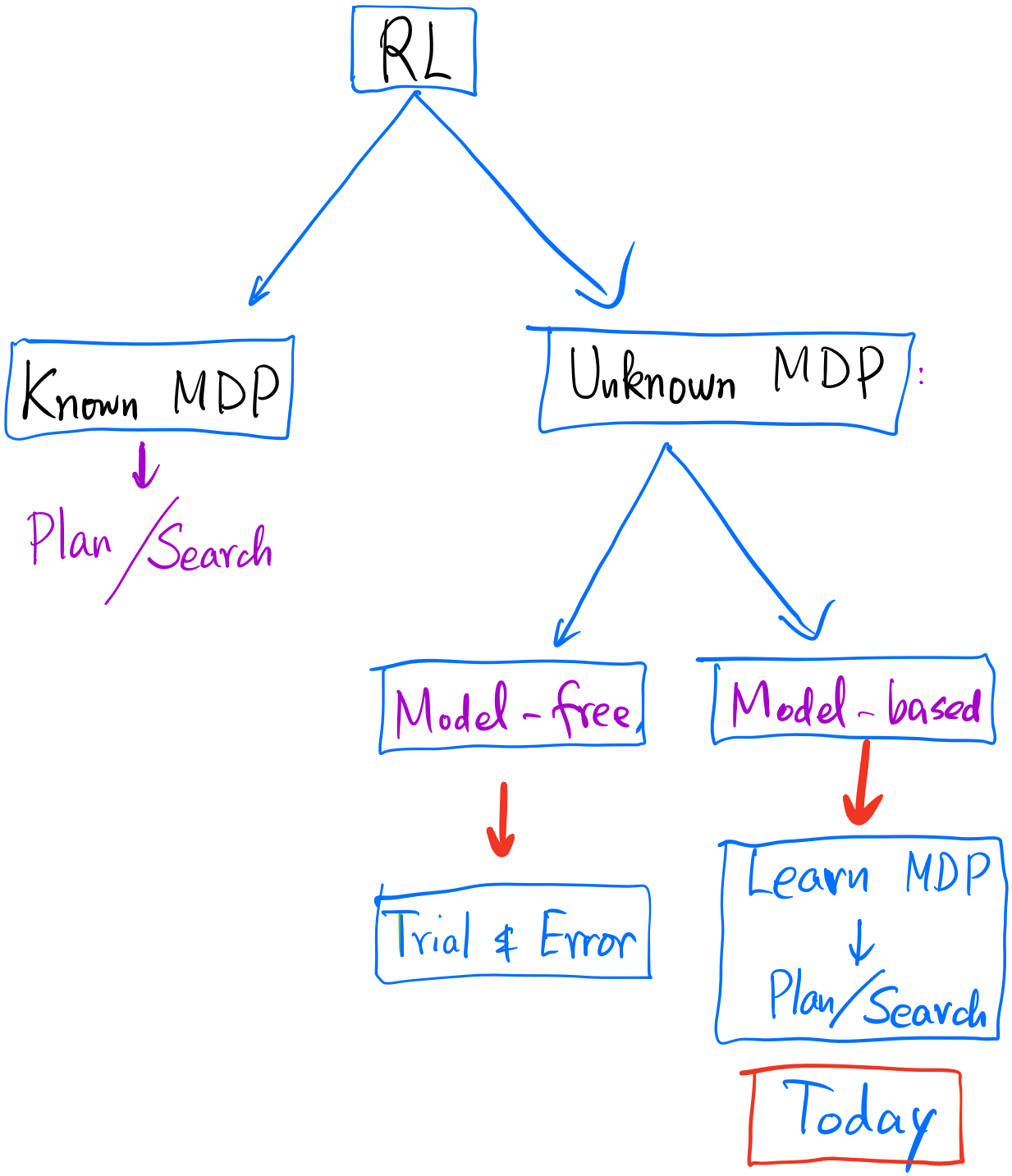
Integrating Learning & Planning

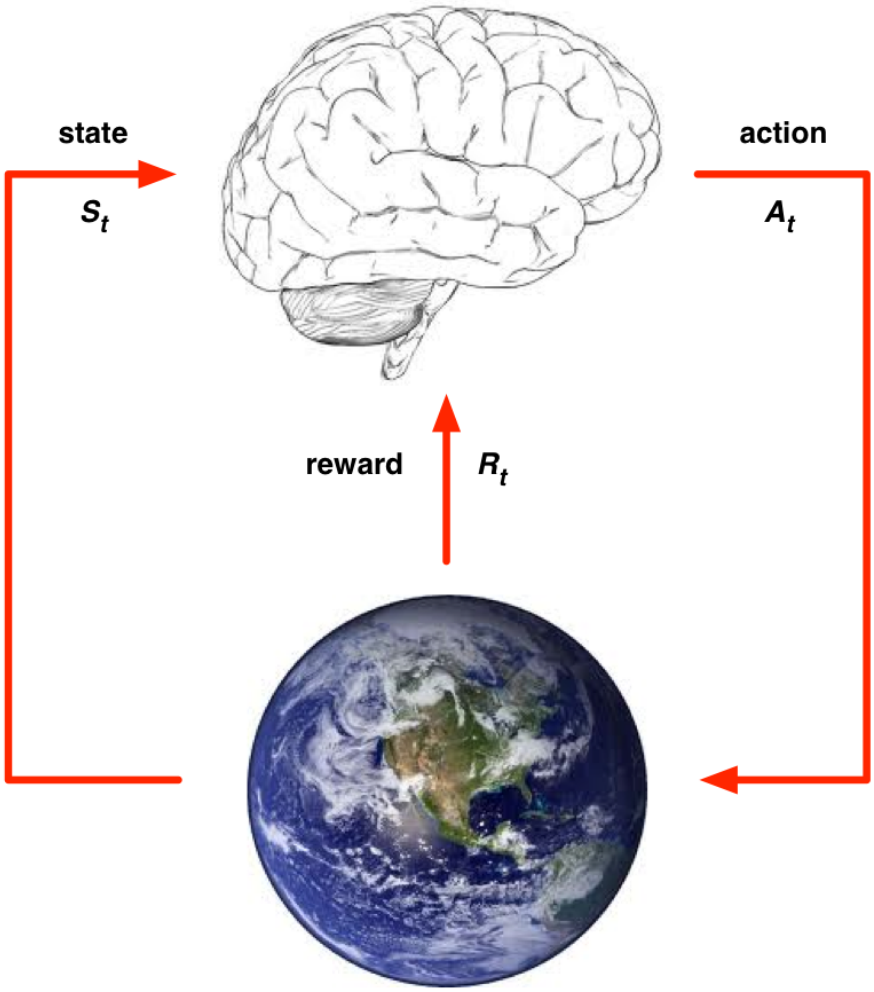
Agenda

- Introduction
- Model-based R.L.
- Integrated Architectures
- Simulation-Based Search

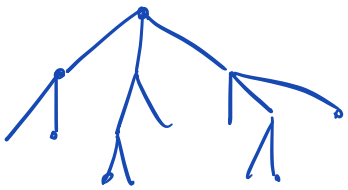
Introduction



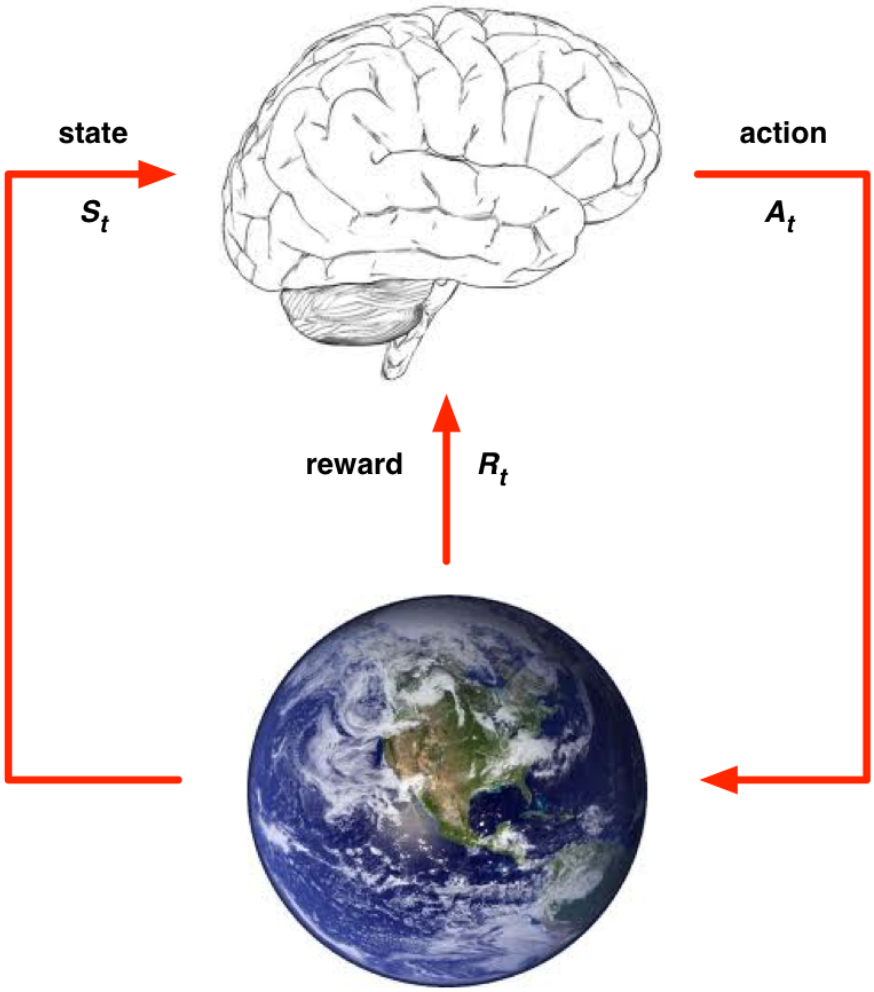
RL with full MDP



MDP : $P(s', r \mid s, a)$ ✓

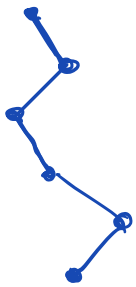
- RL with known MDP: $\{\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}\}$ ✓ ✓
  - e.g. Dynamic Programming
  - Planning = Looking ahead → far into the future
    → for many trajectories.
  - 
  - Model $\xrightarrow{\text{simulated}}$ Experience $\xrightarrow{\text{Backups}}$ Values $\xrightarrow{\text{Improve}}$ Policy

Model-free RL.



What if $P(s', r \mid s, a)$ is unknown?

- RL with unknown MDP: $\{\mathcal{S}, \mathcal{A}\}$ = Model-free RL
  - e.g. Monte-Carlo, TD Learning ($v_\pi(s)$)
    SARSA, Q-Learning ($q_\pi(s,a)$)
  - Experience → value → policy, or → policy
  - Learning = Looking ahead → one step at a time; one trajectory at a time



  - Real Experience —Backups→ Values —Improve→ Policy

Common features

- Calculate $V_\pi(S_t)$ / $q_\pi(S_t, A_t)$ or $P(A_t|S_t)$
- Look ahead to future values
- Compute a backup value
- Iterative update of value function/policy

Model-based learning

Q: Integrate Planning and Learning ?

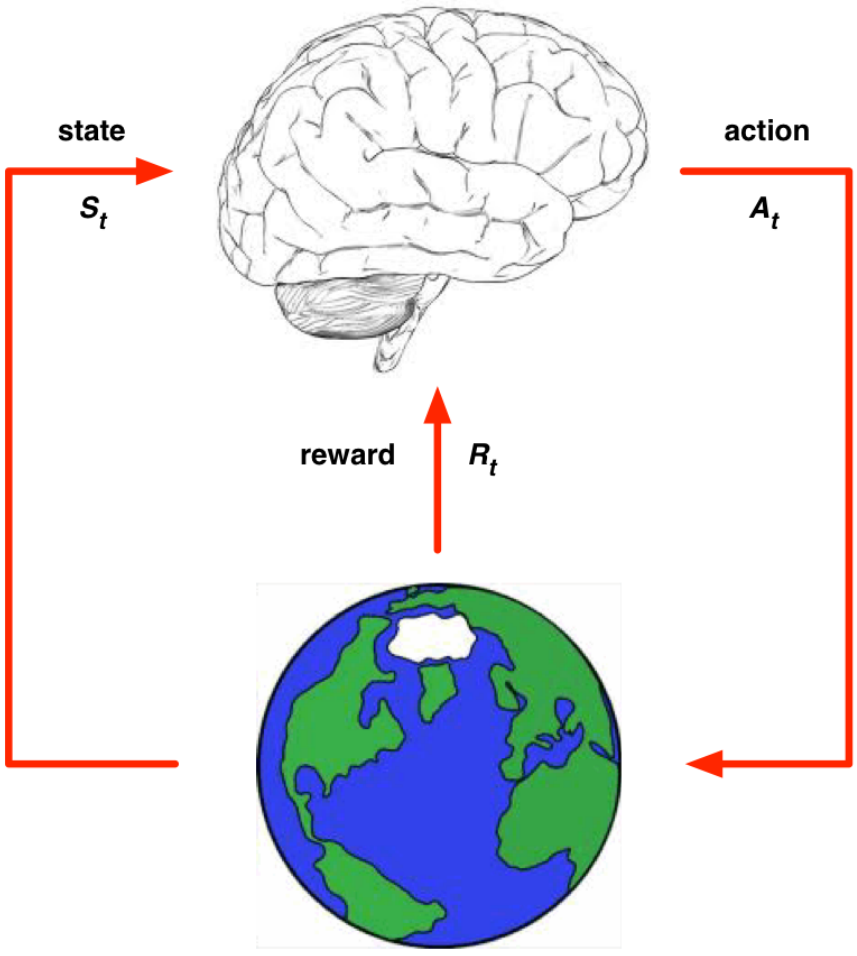
A: YES, but need to

- learn a model from experience
- plan value function/policy from model

⇓

model-based learning

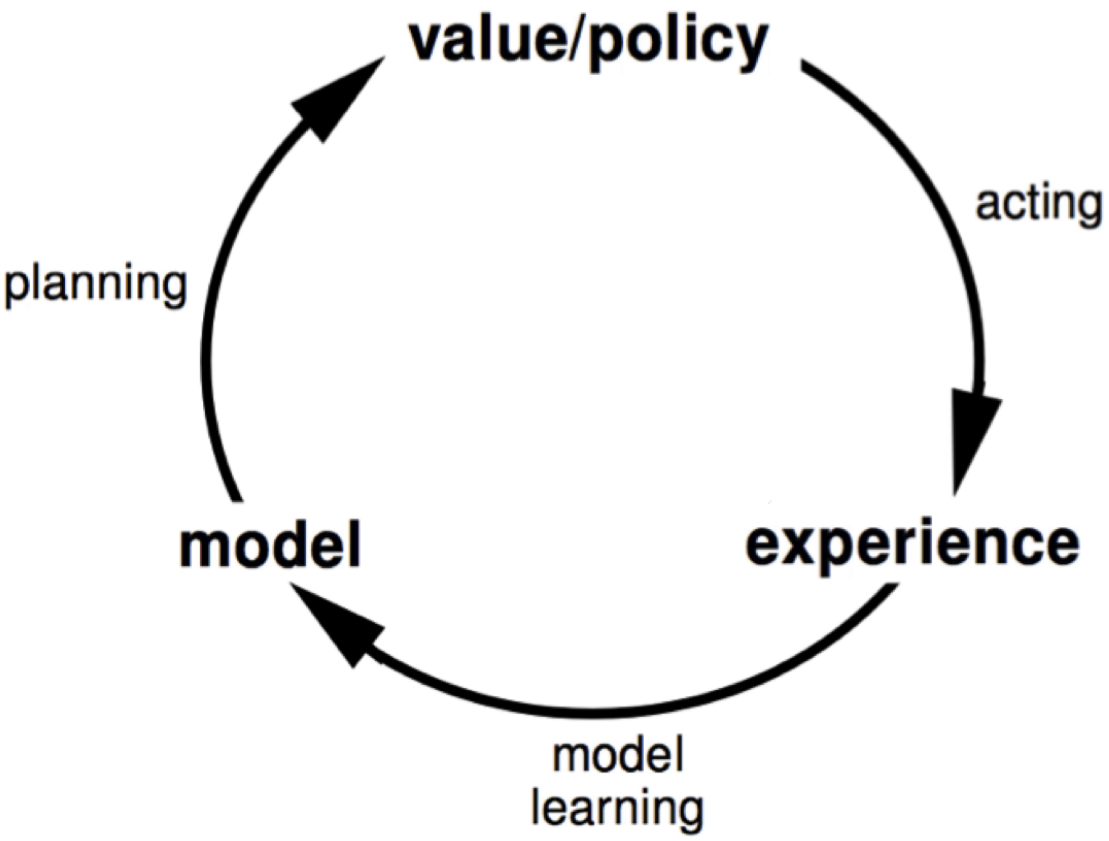
Model-based R.L.



↑

Approximate $\hat{p}(s', r|s, a) \approx p(s', r|s, a)$.

Model-based R.L.

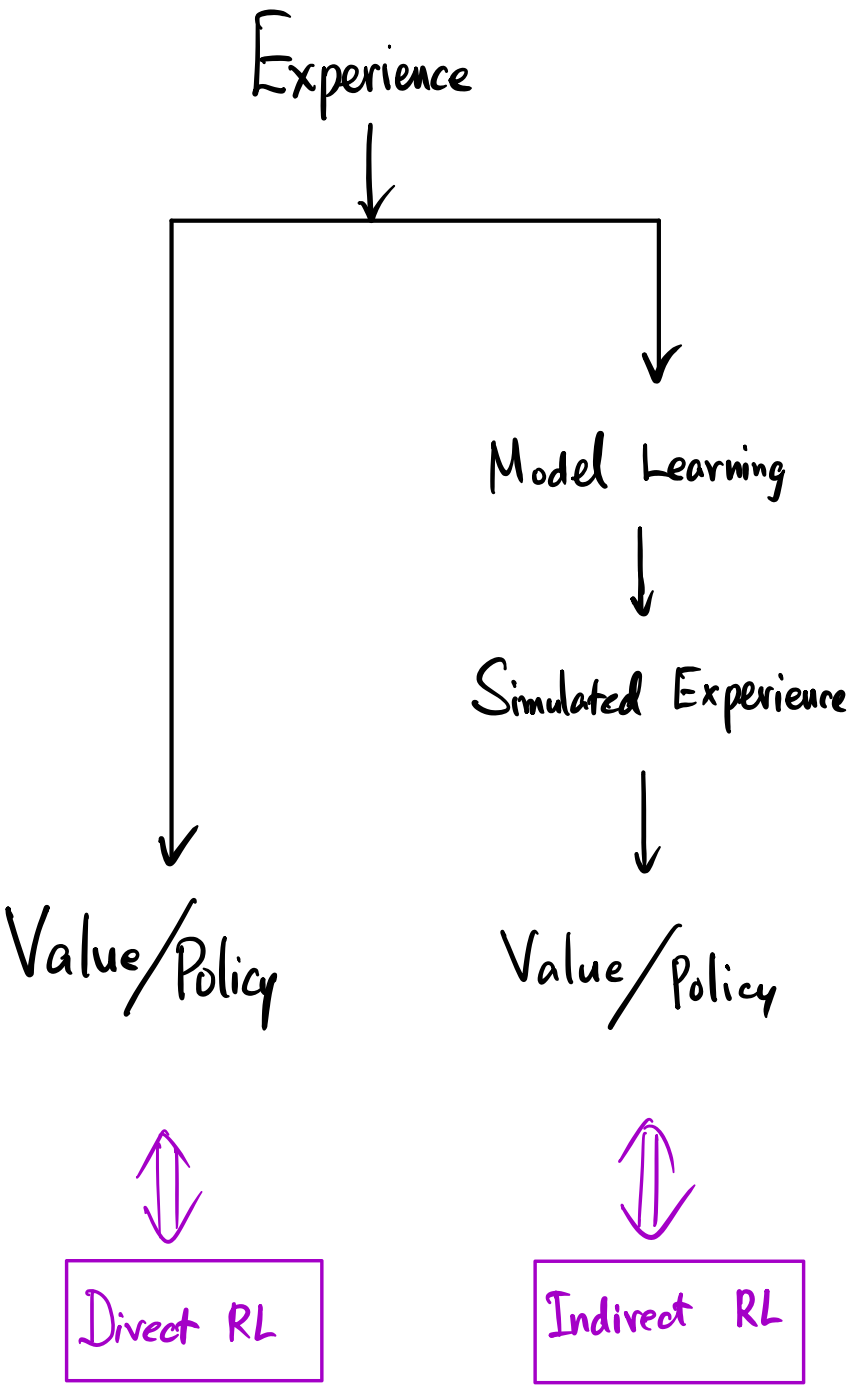


Pros & Cons of Model-based R.L.

- Pros
  - Can learn model efficiently by Supervised Learning
  - Can reason about model uncertainty.
- Cons.

  Two sources of approximation error
  - First learn a model
  - Second construct value function

Direct RL vs Indirect RL



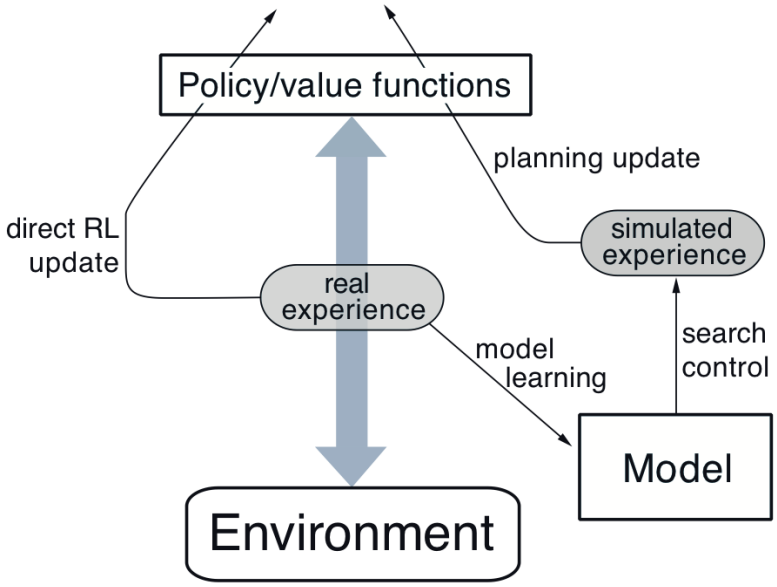


Figure 8.1: The general Dyna Architecture. Real experience, passing back and forth between the environment and the policy, affects policy and value functions in much the same way as does simulated experience generated by the model of the environment.

- Comparison:

Direct RL	vs	Indirect RL.
Trial & Error		Reasoning = Cognition
Reactive Decision Making		Deliberative Planning
Little computation		Computationally intensive

- How do humans learn?

  Direct + Indirect ?

- Q: RL: { Integrate both ?
  How ?

Model learning

Model Representation

- True model $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$ (✓ ✓ ? ?)

- Representation: $\mathcal{M}_\eta = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}_\eta, \mathcal{R}_\eta \rangle$

$$S_{t+1} \sim \mathcal{P}_\eta(S_{t+1} \mid S_t, A_t)$$

$$R_{t+1} = \mathcal{R}_\eta(R_{t+1} \mid S_t, A_t)$$

- Typically, assume $S_{t+1} \perp\!\!\!\perp R_{t+1} \mid S_t, A_t$:

$$\mathcal{P}_\eta(S_{t+1}, R_{t+1} \mid S_t, A_t) = \mathcal{P}_\eta(S_{t+1} \mid S_t, A_t)\, \mathcal{P}_\eta(R_{t+1} \mid S_t, A_t)$$

Learn them separately !

Examples of models

- Table Lookup
- Linear Expectation
- Basis expansion (e.g. Linear Gaussian)
- Decision Trees
- Neural Network, e.g. CNN, DNN.
- ... ...

Model Learning

- Experience

$$\{S_1, A_1, R_2, S_2, \cdots\cdots S_T\}$$

- Data :

$$(S_1, A_1) \rightarrow (R_2, S_2)$$
$$(S_2, A_2) \rightarrow (R_3, S_3)$$
$$\vdots$$
$$(S_{T-1}, A_{T-1}) \rightarrow (R_T, S_T)$$

(Implicitely used Markov assumption).

- Learning η:

$$S_{t+1} \sim p_\eta(S_{t+1} \mid S_t, A_t)$$

$$R_{t+1} = R_\eta(R_{t+1} \mid S_t, A_t)$$

  - Loss function

    L_2 or KL divergence

  - $\hat{\eta} = \underset{\eta}{\operatorname{argmin}}$ Empirical Loss.

e.g. Table Lookup Model

- Note

$$p(s', r \mid s, a) = \Pr\left(R_{t+1} = r,\ S_{t+1} = s' \mid S_t = s,\ A_t = a\right)$$

$$= \frac{\Pr\left((S_t, A_t, R_{t+1}, S_{t+1}) = (s, a, r, s')\right)}{\Pr\left((S_t, A_t) = (s, a)\right)}$$

In particular,

$$p(s' \mid s, a) = \Pr\left(S_{t+1} = s' \mid S_t = s,\ A_t = a\right)$$

$$= \frac{\Pr\left((S_t, A_t, S_{t+1}) = (s, a, s')\right)}{\Pr\left((S_t, A_t) = (s, a)\right)}$$

$$r(s, a) = \mathbb{E}\left(R_{t+1} \mid S_t = s,\ A_t = a\right)$$

$$= \frac{\mathbb{E}\left(R_{t+1}\, I\{S_t = s,\ A_t = a\}\right)}{\Pr\left((S_t, A_t) = (s, a)\right)}$$

- Learnt Model

$$\hat{P}(s'|s,a) = \frac{\hat{\Pr}((S_t, A_t, S_{t+1}) = (s,a,s'))}{\hat{\Pr}((S_t, A_t) = (s,a))}$$

$$= \frac{T^{-1}\sum_{t=1}^{T} I\{(S_t, A_t, S_{t+1}) = (s,a,s')\}}{T^{-1}\sum_{t=1}^{T} I\{(S_t, A_t) = (s,a)\}}$$

$$= \frac{\sum_{t=1}^{T} I\{(S_t, A_t, S_{t+1}) = (s,a,s')\}}{N(s,a)}$$

$$\hat{r}(s,a) = \frac{\hat{\mathbb{E}}(R_{t+1} I\{S_t = s, A_t = a\})}{\hat{\Pr}((S_t, A_t) = (s,a))}$$

$$= \frac{T^{-1}\sum_{t=1}^{T} R_{t+1} I\{(S_t, A_t) = (s,a)\}}{T^{-1}\sum_{t=1}^{T} I\{(S_t, A_t) = (s,a)\}}$$

$$= \frac{\sum_{t=1}^{T} R_{t+1} I\{(S_t, A_t) = (s,a)\}}{N(s,a)}$$

AB Example

S R S R S

Two states A, B; no discounting; 8 episodes of experience

$A, 0, B, 0$

$B, 1$

$B, 1$

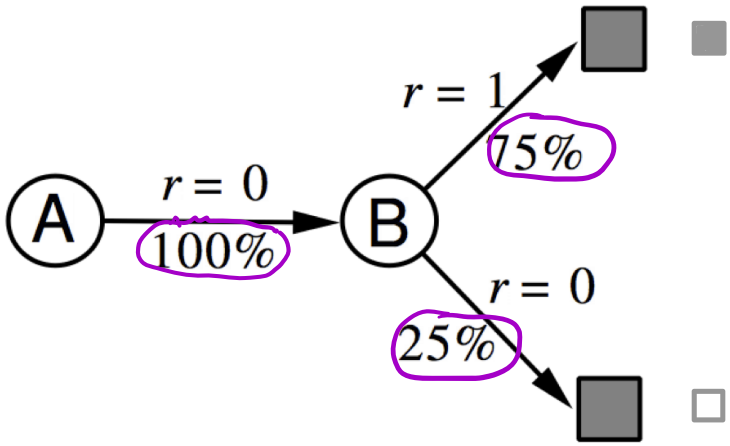
$B, 1$

$B, 1$

$B, 1$

$B, 1$

$B, 0$



We have constructed a table lookup model from the experience

Joint conditional distribution:

$$P(s', r \mid s, a)$$

$$= P(s', r \mid s)$$

$$= \frac{P[(S_t, R_{t+1}, S_{t+1}) = (s, r, s')]}{P(S_t = s)}$$

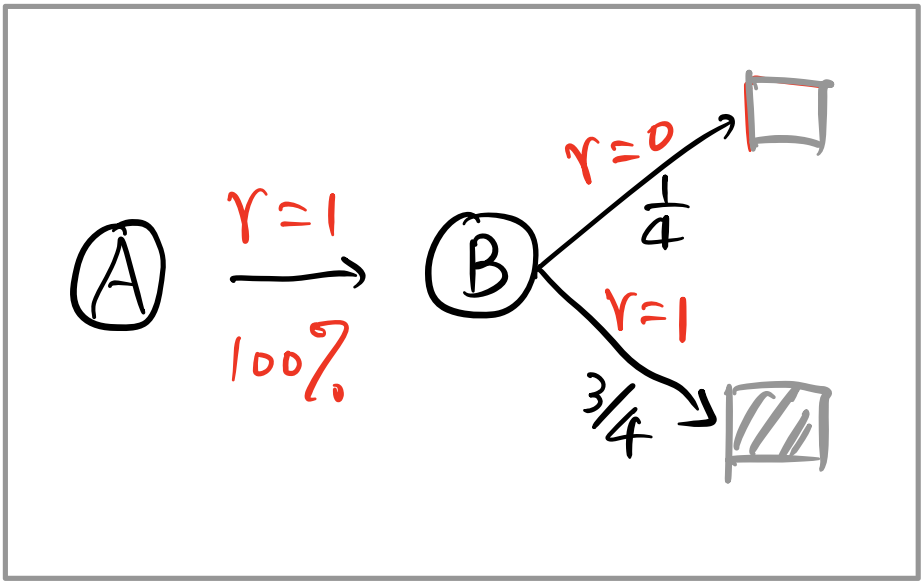
$\Rightarrow$ Marginal conditional dist

Now

$$\hat{P}(B, 0 \mid A) = \frac{\hat{P}(A, 0, B)}{\hat{P}(A)} = 1$$

$$\hat{P}(\square, 0 \mid B) = \frac{\hat{P}(B, 0, \square)}{\hat{P}(B)} = \frac{2}{8} = \frac{1}{4}$$

$$\hat{P}(\blacksquare, 1 \mid B) = \frac{\hat{P}(B, 1, \blacksquare)}{\hat{P}(B)} = \frac{6}{8} = \frac{3}{4}$$



$\Rightarrow$

$$\hat{P}(B|A) = 1, \qquad R|A = 0$$

$$\hat{P}(\cdot|B) = \begin{cases} \frac{1}{4} & \square \\ \frac{3}{4} & \blacksquare \end{cases}$$

- $r|A = 0$

$$r|B = \begin{cases} r(\blacksquare) = 1 & 75\% \\ r(\square) = 0 & 25\% \end{cases}$$

Planning with a model

- Learnt MDP Model

$$\langle S, A, P_\eta, R_\eta \rangle$$

- Use any planning algorithm
  - Value iteration
  - Policy iteration
  - Tree Search
  - ....

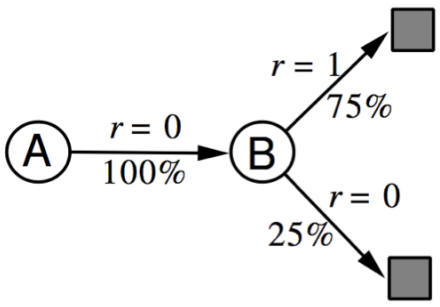
e.g. AB example

Prior: $\hat{\pi}(A) = \frac{1}{8}$, $\hat{\pi}(B) = \frac{7}{8}$

- Construct a table-lookup model from real experience
- Apply model-free RL to sampled experience

Real experience

A, 0, B, 0
B, 1
B, 1
B, 1
B, 1
B, 1
B, 1
B, 0



Sampled experience

B, 1
B, 0
B, 1
A, 0, B, 1
B, 1
A, 0, B, 1
B, 1
B, 0

e.g. Monte-Carlo learning: $V(A) = 1, V(B) = 0.75$

different from that using real experience

Used Markov Property

Sampling

- Step 1 : Sample initial state

Prior : $\hat{\pi}(A) = \frac{1}{8}$, $\hat{\pi}(B) = \frac{7}{8}$

e.g. A. $\Rightarrow$ $r|A = 0$.

- Step 2 : Sample trajectory

Sample next state/reward from table lookup model

- Step 3 : Monte Carlo evaluation

$V(A) = \frac{1+1}{2} = 1$, $V(B) = \frac{6}{8} = 0.75$.

Sample-based planning

- Simple but powerful, efficient.

  Use the model ONLY to sample

- Sample experience from model

$$S_{t+1} \sim P_\eta(S_{t+1} \mid S_t, A_t)$$

$$R_{t+1} = R_\eta(R_{t+1} \mid S_t, A_t)$$

- Apply model-free RL to samples.

  e.g.
  - Monte-Carlo
  - SARSA
  - Q-learning

Integrated Architecture

Real and Simulated Experience

- Real experience

$$S' \sim P(s' | s, a)$$

$$R = R(s, a)$$

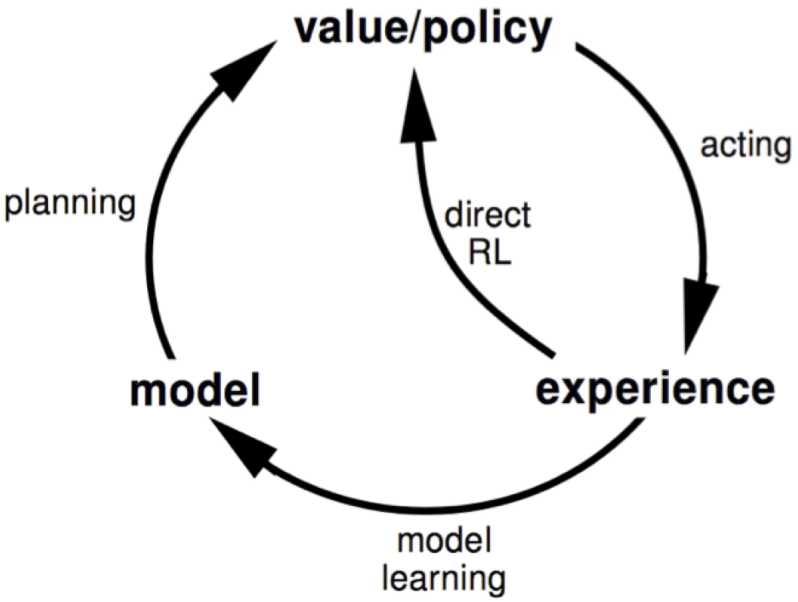
- Simulated experience

$$S' \sim P_\eta(s' | s, a)$$

$$R = R_\eta(s, a)$$

Dyna

Dyna Architecture



Integrating Learning and Planning

- Model-Free RL
  - No model
  - Learn value function (and/or policy) from real experience
- Model-Based RL (using Sample-Based Planning)
  - Learn a model from real experience
  - Plan value function (and/or policy) from simulated experience
- Dyna
  - Learn a model from real experience
  - Learn and plan value function (and/or policy) from real and simulated experience



Dyna-Q Algorithm

Dyna-Q Algorithm

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Do forever:

(a) $S \leftarrow$ current (nonterminal) state

(b) $A \leftarrow \varepsilon$-greedy(S,Q)

(c) Execute action A; observe resultant reward, R, and state, S'

(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)

(f) Repeat n times:

$S \leftarrow$ random previously observed state

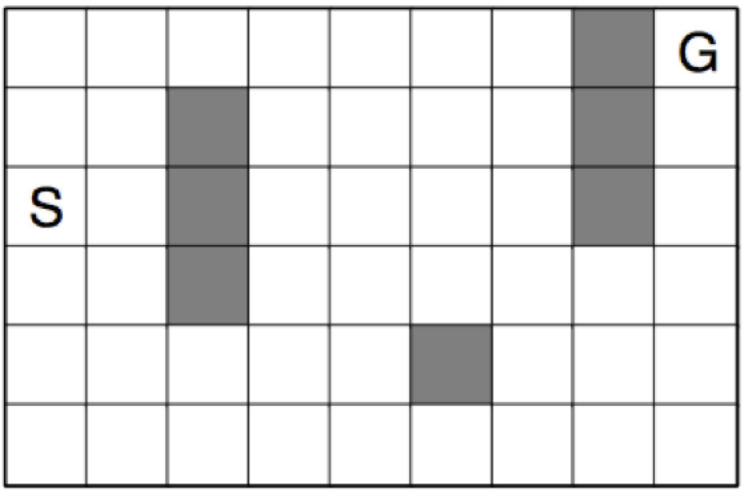
$A \leftarrow$ random action previously taken in S

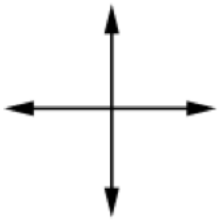
$R, S' \leftarrow Model(S,A)$

$Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

supervised learning.

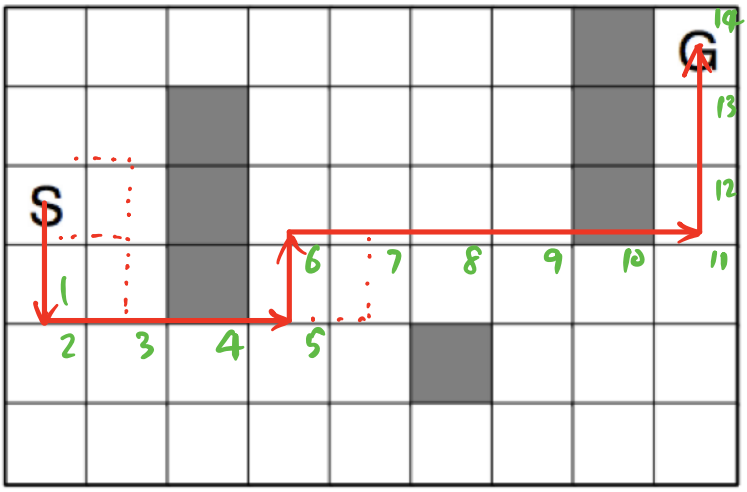
Simple Maze

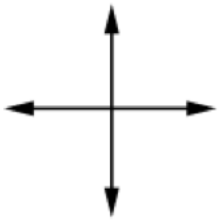




actions

- States / Actions
- $R_t = \begin{cases} 0 & t < T \\ 1 & t = T \end{cases}$
- $\gamma = 0.95$ (encourge fast convergence, otherwise, final reward discounted)
- $\alpha = 0.1$
- $\varepsilon = 0.1$

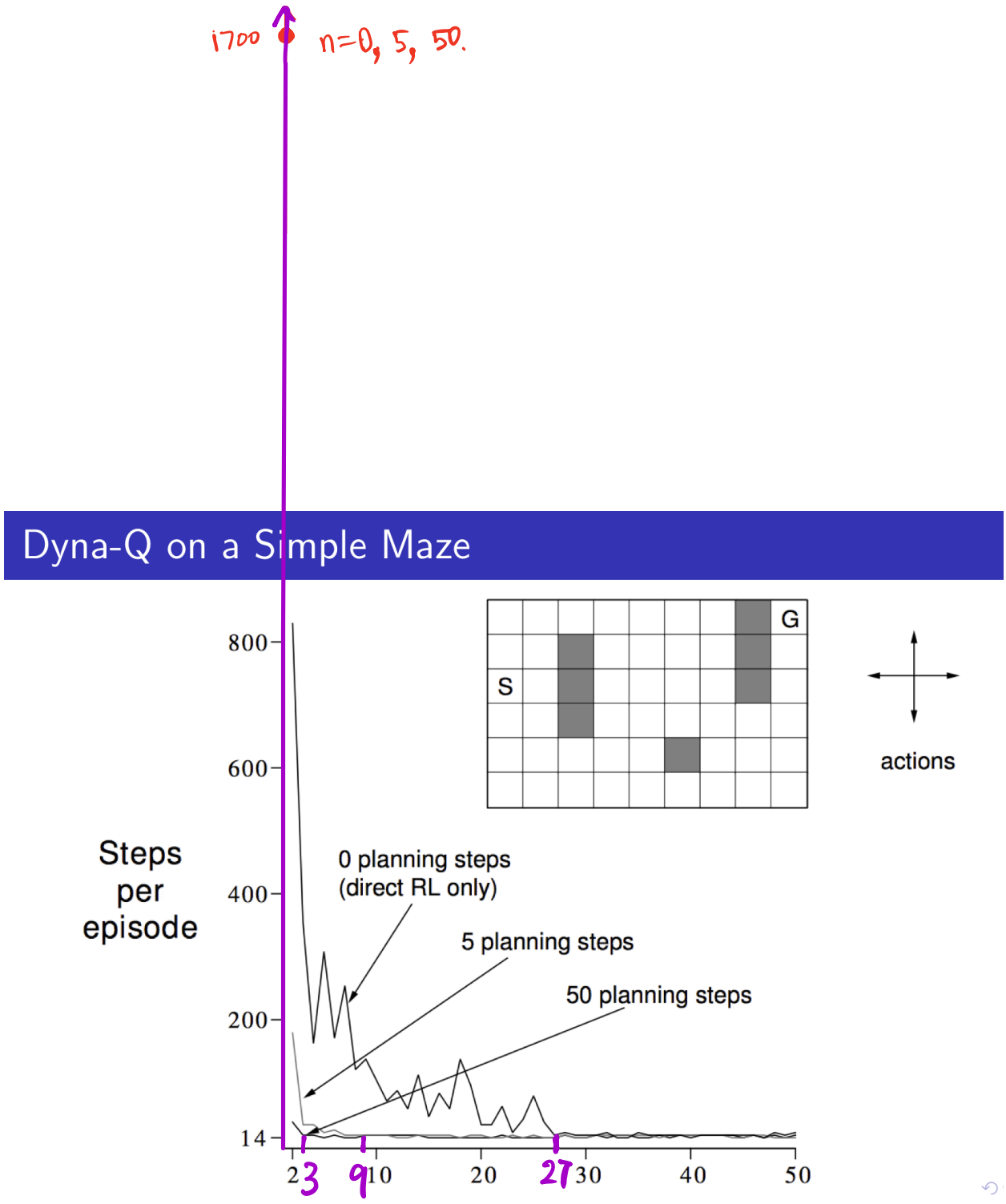




actions

- Optimal policy : above (not unique)

- Optimal # of steps = 14


1700
n=0, 5, 50.
Dyna-Q on a Simple Maze
G
S
actions
Steps per episode
800
600
400
200
14
0 planning steps
(direct RL only)
5 planning steps
50 planning steps
2
3
9
10
20
27
30
40
50

Observations

- Each curve: average of 30 repetitions.
- No planning ($n = 0$):

  First episode takes 1700 steps

  It takes 27 episodes to reach optimal

- Planning is better.

  $n = 5 \implies$ 9 episodes to "optimal"

  $n = 50 \implies$ 3 episodes to "optimal"

- A little planning: reasonably good.

- In practice,

  use as much planning as possible.

Non-Planning vs Planning

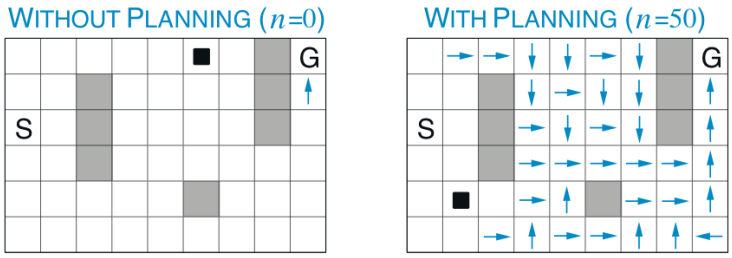
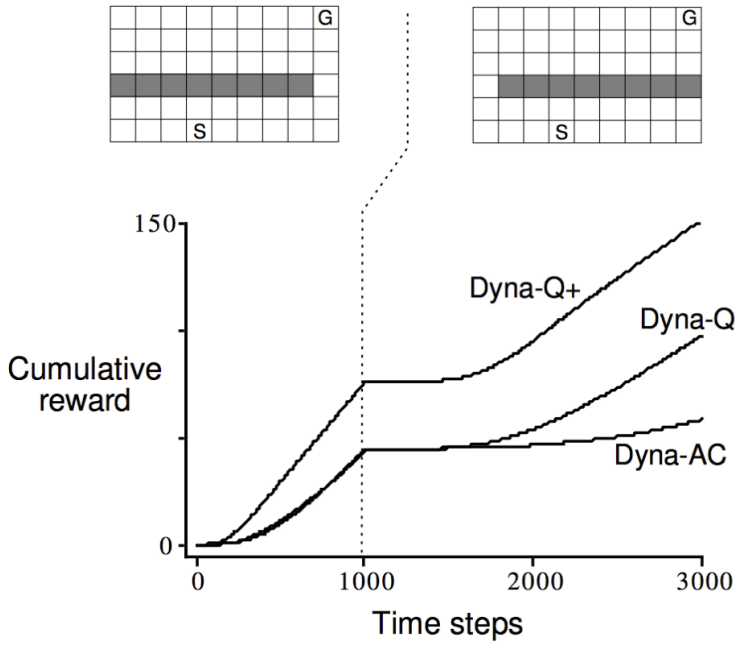


Figure 8.3: Policies found by planning and nonplanning Dyna-Q agents halfway through the second episode. The arrows indicate the greedy action in each state; if no arrow is shown for a state, then all of its action values were equal. The black square indicates the location of the agent. ■

- Non-Planning Dyna (Left)
  - In episode 1, only one step (last) is learnt
  - In episode 2, no progress is made halfway.
- Planning Dyna (Left)
  - In episode 1, only one step (last) is learnt
  - In episode 2, 3/4 of the policy is learnt (good, but not optimal e.g. bottom right) agent is wondering about.
  - In episode 3, the optimal policy is learnt

Dyna-Q with an Inaccurate Model

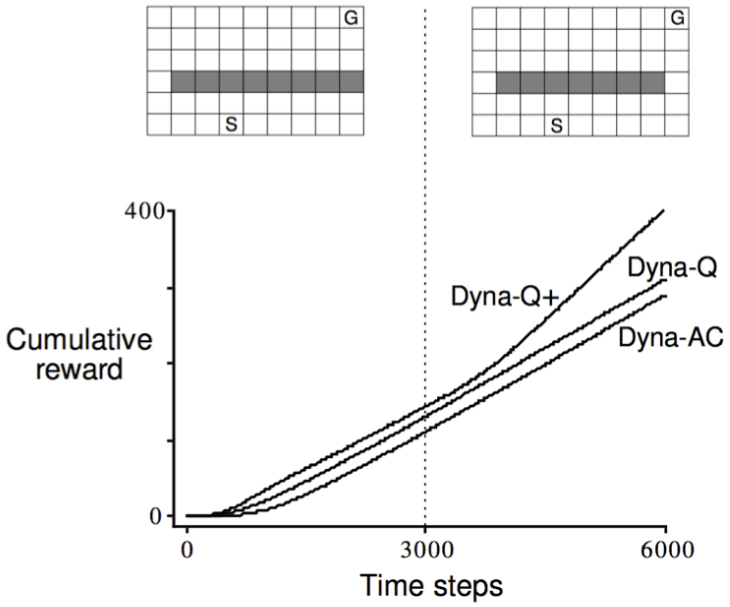
- The changed environment is harder



- Dyna works: Planning ≥ No planning
- Dyna-Q+ ≥ Dyna-Q ≥ Dyna-AC
  - Q-learning effective for small # of state/action
  - Q+ encourages more explorations, and detects changed enviornment faster.

Dyna-Q with an Inaccurate Model (2)

- The changed environment is easier



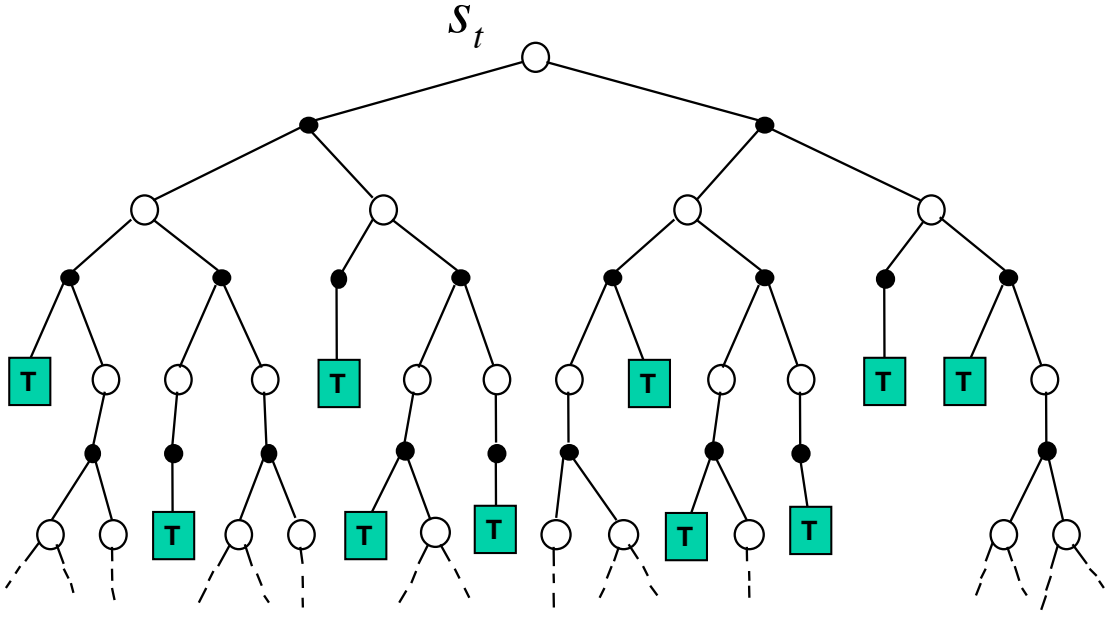
- Dyna-Q+ ≥ Dyna-Q ≥ Dyna-AC
- Q+ encourages more explorations, and detects changed enviornment faster.

- { Nothing is learnt in the Beginning.
  Once something is learnt, it will take off.

Simulation-based Search

Forward Search

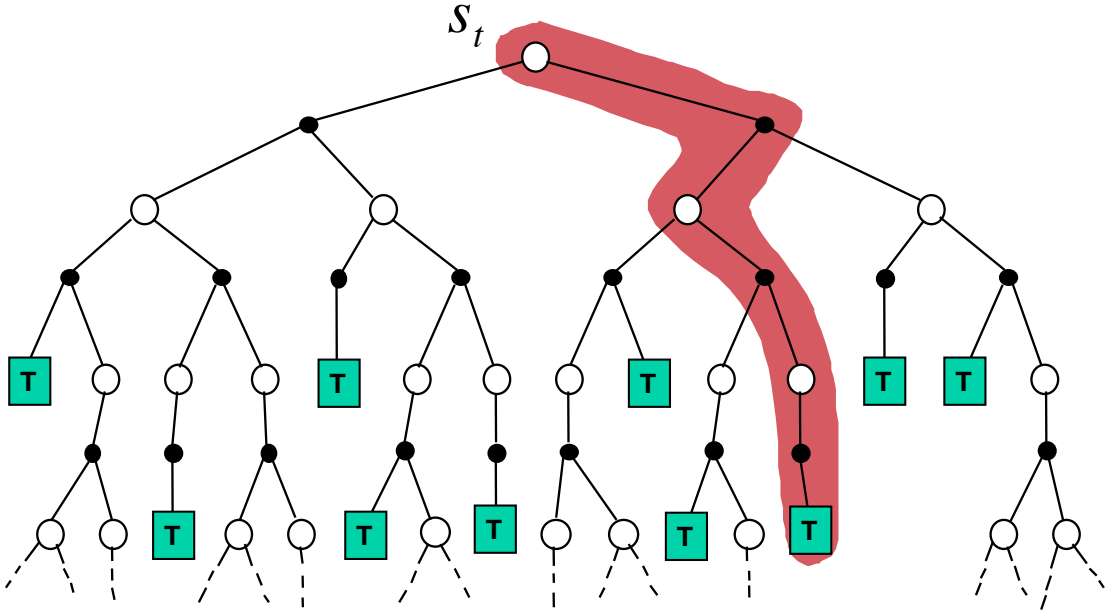
- Forward search algorithms select the best action by lookahead
- They build a search tree with the current state s_t at the root
- Using a model of the MDP to look ahead



- No need to solve whole MDP, just sub-MDP starting from now

Simulation-Based Search

- Forward search paradigm using sample-based planning
- Simulate episodes of experience from now with the model
- Apply model-free RL to simulated episodes



Simulation-Based Search (2)

- Simulate episodes of experience from now with the model

$$\{s_t^k, A_t^k, R_{t+1}^k, ..., S_T^k\}_{k=1}^K \sim \mathcal{M}_\nu$$

- Apply model-free RL to simulated episodes
  - Monte-Carlo control $\rightarrow$ Monte-Carlo search
  - Sarsa $\rightarrow$ TD search

Simple Monte-Carlo Search

- Given a model $\mathcal{M}_\nu$ and a simulation policy π
- For each action $a \in \mathcal{A}$
  - Simulate K episodes from current (real) state s_t

$$\{s_t, a, R^k_{t+1}, S^k_{t+1}, A^k_{t+1}, ..., S^k_T\}^K_{k=1} \sim \mathcal{M}_\nu, \pi$$

  - Evaluate actions by mean return (Monte-Carlo evaluation)

$$Q(s_t, a) = \frac{1}{K} \sum_{k=1}^{K} G_t \xrightarrow{P} q_\pi(s_t, a)$$

- Select current (real) action with maximum value

$$a_t = \underset{a \in \mathcal{A}}{\text{argmax}}\ Q(s_t, a)$$

Monte-Carlo Tree Search (Evaluation)

- Given a model $\mathcal{M}_\nu$
- Simulate K episodes from current state s_t using current simulation policy π

$$\{s_t, A_t^k, R_{t+1}^k, S_{t+1}^k, ..., S_T^k\}_{k=1}^K \sim \mathcal{M}_\nu, \pi$$

- Build a search tree containing visited states and actions
- Evaluate states $Q(s, a)$ by mean return of episodes from s, a

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{k=1}^{K} \sum_{u=t}^{T} \mathbf{1}(S_u, A_u = s, a) G_u \xrightarrow{P} q_\pi(s, a)$$

- After search is finished, select current (real) action with maximum value in search tree

$$a_t = \underset{a \in \mathcal{A}}{\text{argmax}}\ Q(s_t, a)$$

Monte-Carlo Tree Search (Simulation)

- In MCTS, the simulation policy π improves
- Each simulation consists of two phases (in-tree, out-of-tree)
  - Tree policy (improves): pick actions to maximise $Q(S, A)$
  - Default policy (fixed): pick actions randomly
- Repeat (each simulation)
  - Evaluate states $Q(S, A)$ by Monte-Carlo evaluation
  - Improve tree policy, e.g. by $\epsilon -$ greedy(Q)
- Monte-Carlo control applied to simulated experience
- Converges on the optimal search tree, $Q(S, A) \rightarrow q_*(S, A)$

The Game of GO

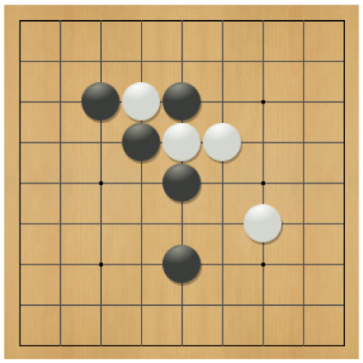
围棋

- The ancient oriental game of Go is 2500 years old
- Considered to be the hardest classic board game
- Considered a grand challenge task for AI *(John McCarthy)*
- Traditional game-tree search has failed in Go



Rules of Go

- Usually played on 19x19, also 13x13 or 9x9 board
- Simple rules, complex strategy
- Black and white place down stones alternately
- Surrounded stones are captured and removed
- The player with more territory wins the game





Position Evaluation in Go

- How good is a position s ?
- Reward (undiscounted)

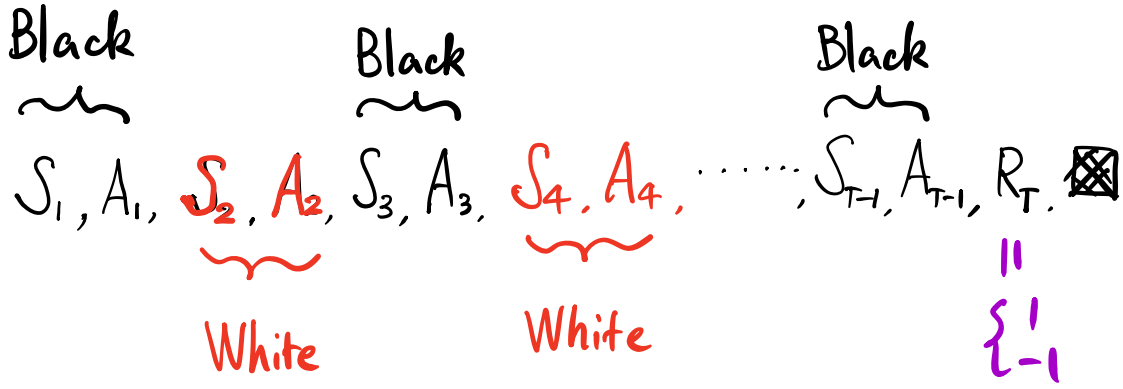
  $R_t = 0$ for all non-terminal steps $t < T$

  $R_T = I\{\text{Black Wins}\}$

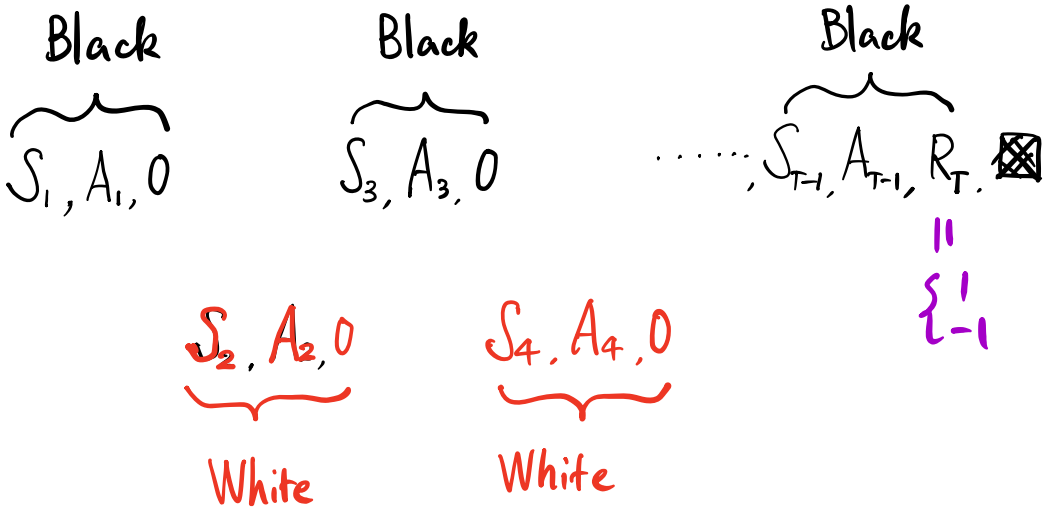
- Policy π_W selects moves for black player.
- Value function:

  $V_{\pi_B}(s) = \mathbb{P}_r\left(\text{Black Wins} \middle| S_t = s\right)$

- One trajectory (episode):



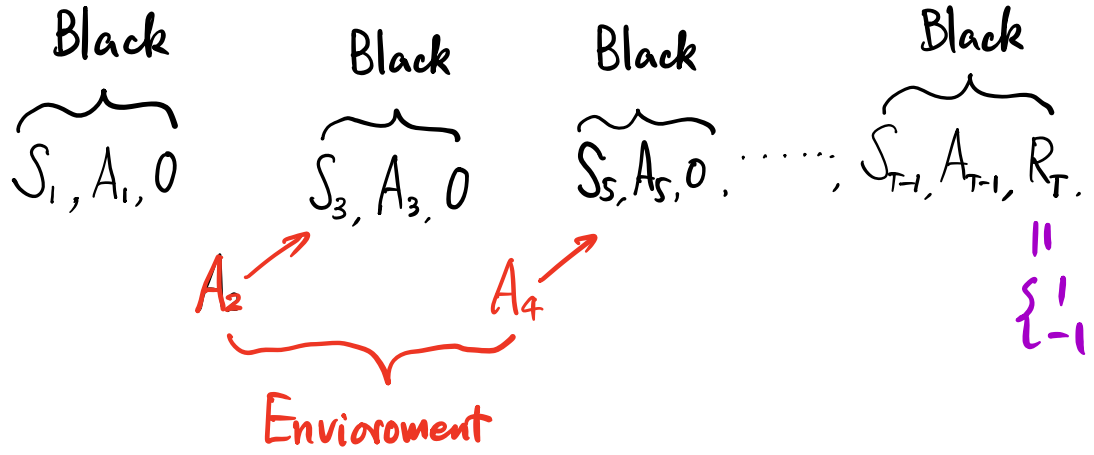
- Separate Black / White moves:



- Deterministic transitions

$$S_{2m} = f(S_{2m-1}, A_{2m-1}), \quad S_{2m+1} = g(S_{2m}, A_{2m})$$

- Focus on Black player:



$\Rightarrow$ "White Actions": environment!

- MDP with unknown model

$$S_1, A_1, 0, S_3, A_3, 0, S_5, A_3, 0, \cdots, S_{T-1}, A_{T-1}, R_T.$$

- Approaches:
  - Model-free
    - Value-based

      (e.g., MC, SARSA, Q-learning)
    - Policy Gradient
    - Actor-Critic
  - Model-based:

TODAY

Problems with Model-free Approach.

- Q: How to generate opponent's actions? (White)
- Randomly generating actions
  - Highly inefficient
  - e.g. illegal actions / stupid moves.

Q: Can we do better?

A: Yes, but how?

- Supervised learning:

  Learn from the past human games

  $\Rightarrow$ AI level → High ✓

  AI level → Superhuman ✗

- Nash equilibrium (Best response)

  i.e., Both players aspire to be top players.

Position Evaluation in Go

- How good is a position s?
- Reward function (undiscounted):

$$R_t = 0 \text{ for all non-terminal steps } t < T$$

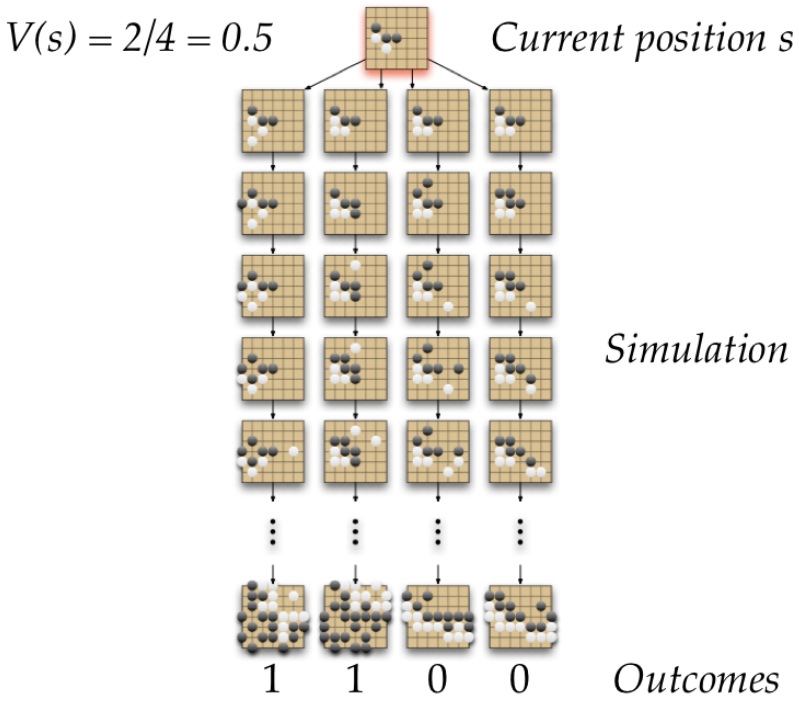
$$R_T = \begin{cases} 1 & \text{if Black wins} \\ 0 & \text{if White wins} \end{cases}$$

- Policy $\pi = \langle \pi_B, \pi_W \rangle$ selects moves for both players
- Value function (how good is position s):

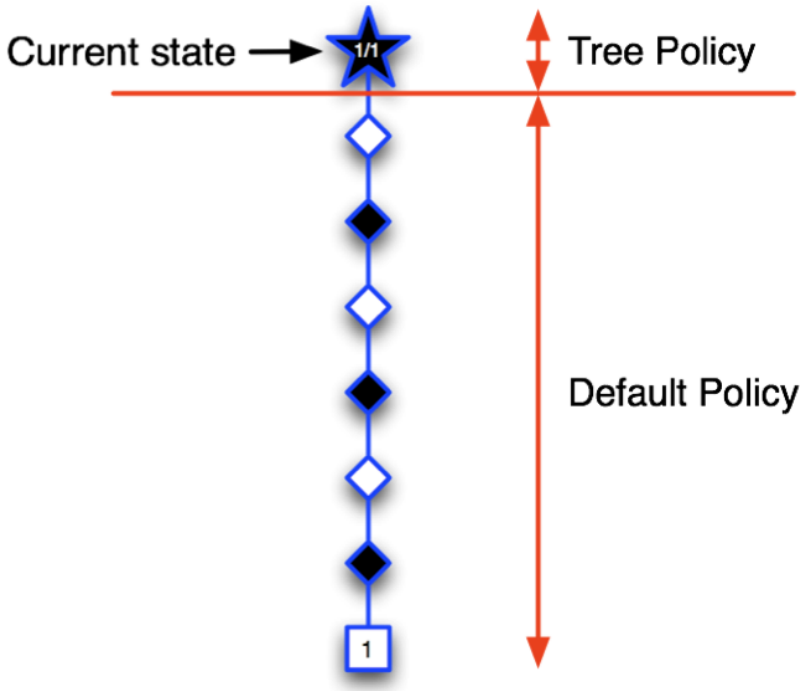
$$v_\pi(s) = \mathbb{E}_\pi \left[R_T \mid S = s \right] = \mathbb{P} \left[\text{Black wins} \mid S = s \right]$$

$$v_*(s) = \max_{\pi_B} \min_{\pi_W} v_\pi(s)$$

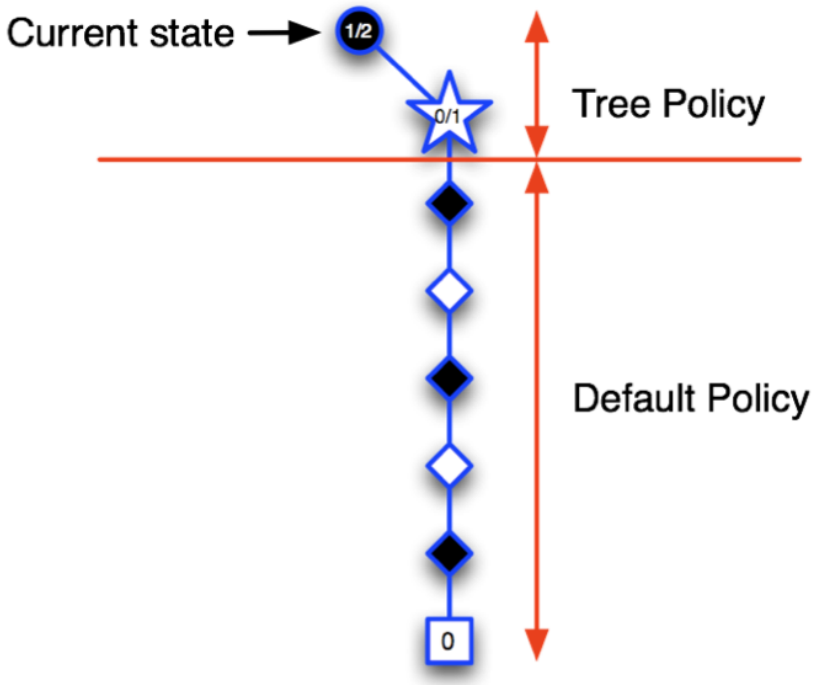
Monte-Carlo Evaluation in Go



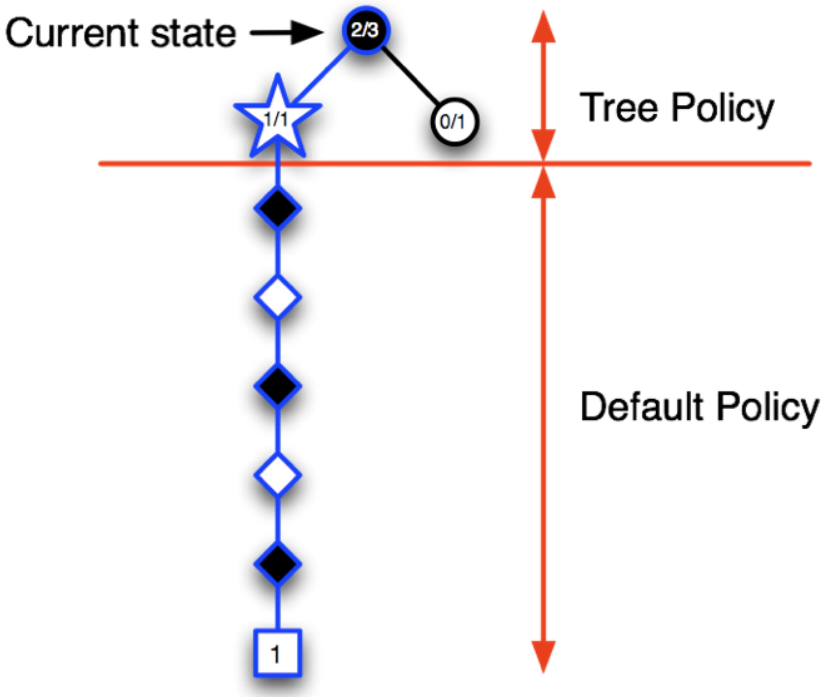
Applying Monte-Carlo Tree Search (1)



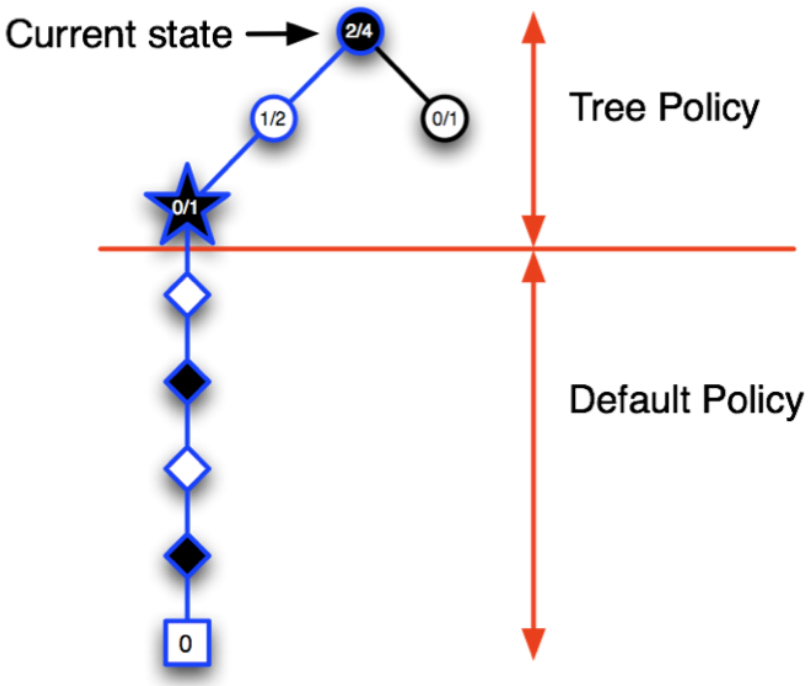
Applying Monte-Carlo Tree Search (2)



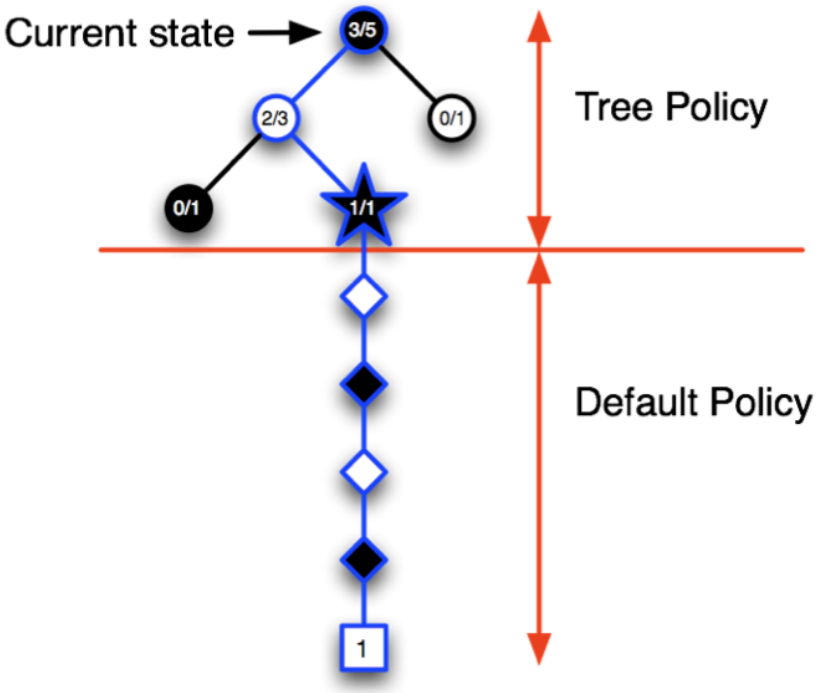
Applying Monte-Carlo Tree Search (3)



Applying Monte-Carlo Tree Search (4)



Applying Monte-Carlo Tree Search (5)



Advantages of MC Tree Search

- Highly selective best-first search
- Evaluates states *dynamically* (unlike e.g. DP)
- Uses sampling to break curse of dimensionality
- Works for "black-box" models (only requires samples)
- Computationally efficient, anytime, parallelisable

Temporal-Difference Search

- Simulation-based search
- Using TD instead of MC (bootstrapping)
- MC tree search applies MC control to sub-MDP from now
- TD search applies Sarsa to sub-MDP from now

MC vs. TD search

- For model-free reinforcement learning, bootstrapping is helpful
  - TD learning reduces variance but increases bias
  - TD learning is usually more efficient than MC
  - TD(λ) can be much more efficient than MC
- For simulation-based search, bootstrapping is also helpful
  - TD search reduces variance but increases bias
  - TD search is usually more efficient than MC search
  - TD(λ) search can be much more efficient than MC search

TD Search

- Simulate episodes from the current (real) state s_t
- Estimate action-value function $Q(s, a)$
- For each step of simulation, update action-values by Sarsa

$$\Delta Q(S, A) = \alpha(R + \gamma Q(S', A') - Q(S, A))$$

- Select actions based on action-values $Q(s, a)$
  - e.g. ϵ-greedy
- May also use function approximation for Q

Dyna-2

- In Dyna-2, the agent stores two sets of feature weights
  - Long-term memory
  - Short-term (working) memory
- Long-term memory is updated from real experience using TD learning
  - General domain knowledge that applies to any episode
- Short-term memory is updated from simulated experience using TD search
  - Specific local knowledge about the current situation
- Over value function is sum of long and short-term memories

Results of TD search in Go

